

Pillar 1 — Portfolio Analytics

Data Architecture & Phased Build Plan — Reference Document

1. Design Principles

- **Transactions and prices are the atomic source of truth**, not custodian position snapshots. TWR needs true daily valuation; positions are derived by rolling transactions forward and marking to price history. Custodian snapshots are used only for reconciliation.
- **Canonical schema decoupled from custodian format**. Schwab, Fidelity, and Pershing each have proprietary export formats. A thin per-custodian adapter maps raw exports into one canonical schema — the only place custodian-specific logic lives.
- **Different data structures for different entity types**. Reference/config entities (Account, SecurityMaster, BenchmarkDefinition) use Pydantic models for validation; bulk time series (transactions, prices, returns) use pandas DataFrames with an enforced column/dtype contract.
- **Audit hook built in from day one**. Every calculation module writes a CalculationRun record (input hash, params, timestamp, output reference, code version) as a side effect — retrofitting this after Pillars 1 and 2 exist would mean a rebuild.
- **One shared taxonomy across all three pillars**. Asset class / sector / geography live once on SecurityMaster, so Pillar 2's drift monitoring can reuse the exact same classification Pillar 1 uses for allocation and attribution.

2. Custodian Data Format — What to Expect

Standardized *within* a custodian, not *across* custodians. Every Schwab RIA client gets the same export layout, transaction codes, and file structure — customization happens once per custodian, not once per client. But Schwab, Fidelity, and Pershing each use their own proprietary schema, which is exactly why a market of data-orchestration vendors (BridgeFT, Dispatch) exists solely to normalize across them.

- Build the Schwab adapter first, Fidelity second — these dominate the \$50M–\$500M independent RIA segment. Pershing is lower priority.
- A custodian's own format can drift over time (e.g. the Schwab / TD Ameritrade Institutional conversion changed file formats) — expect occasional adapter maintenance, not a one-and-done build.
- Revisit buy-vs-build on the ingestion layer (BridgeFT, Dispatch) once real client export files are in hand and the edge cases are visible.

3. Reference / Config Entities (Pydantic)

Low-cardinality, relationship-bearing entities, validated at creation time. Core models: **Household**, **Account**, **SecurityMaster**, **BenchmarkDefinition**, **InvestmentPolicy** (Pillar 2 stub) and **CalculationRun** (Pillar 3 audit hook) — all defined now so the schema doesn't change later.

```
class SecurityMaster(BaseModel):
    security_id: str
    cusip: Optional[str] = None
    ticker: Optional[str] = None
```

```

name: str
security_type: SecurityType
asset_class: AssetClass
sub_asset_class: Optional[str] = None
sector: Optional[str] = None
geography: Optional[str] = None
expense_ratio: Optional[Decimal] = None

class CalculationRun(BaseModel):
    run_id: UUID
    timestamp: date
    module: str
    account_id: str
    input_hash: str
    params: dict
    output_ref: str
    code_version: str

```

4. Time Series Entities (DataFrame Contracts)

DataFrame	Key columns	Notes
transactions	account_id, security_id, trade_date, txn_type, quantity, price, net_amount	Primary source of truth. Decimal, not float.
prices	security_id, date, close_price, adj_close	adj_close needed for total return, not just price return.
custodian_positions	account_id, as_of_date, security_id, quantity, market_value	Raw snapshot — reconciliation only, never used directly in return calcs.
daily_positions	account_id, security_id, date, quantity, market_value	Derived: transactions rolled forward, marked to prices.
cash_flows	account_id, date, amount, flow_type	Derived: external flows only, filtered from transactions. Feeds MWR/XIRR.
returns	entity_id, entity_type, date, period_return, cumulative_return	Long/tidy format — one table for account, composite, and benchmark returns.
benchmark_prices	benchmark_id, date, level, total_return_level	Composite benchmark constructed from BenchmarkDefinition.

5. Phased Build Plan

Ordered by dependency. Phases 0–4 are the foundation and warrant the most design care — everything from Phase 5 onward is comparatively mechanical once the foundation is solid.

Phase 0 — Custodian Ingestion

Goal: a pure function per custodian — raw export → canonical `transactions_df` + `custodian_positions_df`.

- Put transaction-type mapping (e.g. Schwab “Reinvest Dividend” → DIV) in a config file, not code.
- Decide corporate action handling explicitly — detect known split-like codes, flag unrecognized rows for manual review rather than letting them corrupt `daily_positions`.
- Use Decimal, not float, throughout — fractional shares and DRIP transactions accumulate float error fast.
- Test fixtures should include the annoying cases: fee-only transactions, fractional-share DRIP, same-day buy/sell, inter-account transfers within a household.

“Done” looks like: adapter takes a raw Schwab or Fidelity file and produces `transactions_df` + `custodian_positions_df` that pass Pydantic validation with zero schema violations.

Phase 1 — Security Master & Reference Data

- Key by a stable identifier with a fallback chain — CUSIP first (tickers can change on corporate actions), ticker as fallback.
- Never let an unclassified security default silently to “other” — route to a manual-review queue instead.
- ETFs/equities can use automated ticker-to-sector/geography lookup; mutual funds often need Morningstar-style category mapping instead.

“Done” looks like: every distinct security appearing in `transactions` or `custodian_positions` has a non-null `asset_class` — a hard gate, since Phases 7 and 8 break silently on gaps here.

Phase 2 — Valuation Engine (`daily_positions`)

- Handle missing prices deliberately (holidays, halts, thin trading) — forward-fill with a capped window, then flag a data gap rather than silently stale-pricing.
- Reconciliation against `custodian_positions` is the real test of this phase — flag divergences beyond tolerance. This reconciliation flag is itself audit-trail evidence for Pillar 3.
- Handle differing pricing cadence explicitly: equities/ETFs price daily close, mutual funds price once daily via NAV.

“Done” looks like: derived market values reconcile to custodian-reported values within tolerance across a full year of history before this layer is trusted for TWR.

Phase 3 — Cash Flow Extraction

- Decide account-level vs. household-level MWR up front — an inter-account transfer within a household may or may not count as an “external” flow depending on which level you’re computing.
- Fees should not appear as cash flows — they’re already reflected in ending value; double-counting distorts the return.

- Use the actual effective date of the flow, not a batch-processing date, for XIRR accuracy.

"Done" looks like: *cash_flows_df is ready to feed an XIRR solver with correct signs and accurate dates.*

Phase 4 — Returns Engine (the keystone module)

- TWR: chain-link true daily returns from Phase 2's daily valuations — more accurate than the Modified Dietz approximation many smaller platforms fall back on.
- MWR/IRR: solve via Newton-Raphson / brentq against cash_flows_df + ending value.
- Decide gross vs. net of fees now — fee drag analysis needs both, and this is expensive to retrofit once downstream modules exist.

"Done" looks like: *the engine reproduces a hand-verified example (known cash flows, known correct TWR/IRR) to the decimal place.*

Phase 5 — Benchmark Engine

- Composite benchmarks (e.g. 60/40) need their own rebalancing logic, modeled as a return series construction — not a real portfolio with real transactions.
- Align benchmark and account valuation dates exactly, accounting for market holidays.
- Schema already supports single-index and composite benchmarks via BenchmarkDefinition.components — don't special-case the single-index path.
- Use total return index data (dividends reinvested), not price-only, or the comparison structurally understates the benchmark.

"Done" looks like: *composite benchmark return matches a hand-calculated or published blended return within a small tolerance.*

Phase 6 — Period Return Aggregation

- MTD/QTD/YTD are date-range slices of the cumulative return series, re-based to period start.
- Decide annualized vs. cumulative for 1Y/3Y/5Y (industry convention: annualized beyond one year) and apply consistently.
- Since-inception should track from when the account started being managed on the platform, not necessarily the custodian account-open date.
- Decide and standardize how sub-one-year-history accounts report unavailable periods (N/A vs. partial-period).

"Done" looks like: *any period label can be pulled for any account/composite and returns a correctly annualized answer with no per-period special-casing.*

Phase 7 — Portfolio Construction Analysis

Can be built in parallel with Phase 6 — depends only on daily_positions + SecurityMaster, no returns-engine dependency. Good early demo-value module.

- Allocation breakdown: group daily_positions market value by asset_class / sector / geography as of any date.

- Concentration risk flags: thresholds (e.g. single position >10%, single sector >25%) should be configurable, not hardcoded — and a natural first place to start writing CalculationRun records.
- Fee drag: pull expense ratios from SecurityMaster and advisor fees from account/fee-schedule config; show both percentage and dollar terms.

"Done" looks like: allocation percentages sum to 100% across each grouping dimension; concentration flags fire on exactly the expected test positions.

Phase 8 — Attribution

- Brinson-Fachler decomposition (allocation, selection, interaction effects) is the standard methodology for asset-class/sector attribution — pick it deliberately rather than improvising.
- Holding-level attribution needs per-security returns and weights, not just per-group — a small addition at Phase 4 if not already built in.
- Multi-period attribution has a real smoothing problem (arithmetic effects don't cleanly compound); single-period attribution per standard period is a reasonable MVP starting point.

"Done" looks like: portfolio return minus benchmark return equals the sum of allocation + selection + interaction effects exactly, for a hand-verified test case.

Phase 9 — Risk Metrics Core

- Volatility: annualized std dev of periodic returns — apply the correct scaling factor ($\sqrt{252}$ daily, $\sqrt{12}$ monthly) consistently.
- Sharpe/Sortino need a risk-free rate series (e.g. 3-month T-bill) — a new small external data dependency worth sourcing once.
- Beta: regression of account returns against benchmark returns (already date-aligned from Phase 5).
- Max drawdown: also capture drawdown duration (time to recover), not just magnitude.
- Tracking error: annualized std dev of (account return – benchmark return).

"Done" looks like: each metric matches a reference implementation or hand calc for at least one test account.

Phase 10 — Correlation Matrix

- Build once, reuse in Phases 11, 13, and 14.
- Lookback window choice (1yr/3yr/5yr) meaningfully changes the matrix — correlations are notoriously unstable in stress periods (relevant context for Phase 12's scenario narrative).
- Handle short-history securities with a defined fallback (asset-class proxy or pairwise overlapping-history correlation) rather than crashing or silently dropping positions.

"Done" looks like: matrix is positive semi-definite and shows sensible correlations for known relationships (e.g. two S&P 500 funds near 1.0).

Phase 11 — VaR / CVaR

- Historical VaR: empirical percentile of historical returns_df — no correlation matrix required, but only reflects scenarios that actually occurred in the lookback window.

- Parametric VaR: analytic, using volatility from the correlation matrix — faster, but the normality assumption understates tail risk (a known, worth-stating limitation).
- CVaR/Expected Shortfall: average loss beyond the VaR threshold — increasingly preferred by regulators over VaR alone.
- Report historical and parametric side by side rather than picking one.

"Done" looks like: *historical and parametric VaR are within a sensible range of each other for a normally-behaved test portfolio.*

Phase 12 — Scenario & Stress Testing

- Historical scenarios (2008, 2020, 2022, 2000): apply today's asset-class/sector weights to the actual realized asset-class returns of the historical window — avoids needing individual-security history back to periods before some holdings existed.
- Hypothetical shocks: need equity beta and fixed-income duration sensitivities per asset class (from SecurityMaster or asset-class proxy).
- Factor shocks overlap with Phase 13 — consider sequencing after the Fama-French regression exists.
- High GTM value, relatively low complexity — “what would 2008 do to your portfolio today” is a strong, easy-to-explain deliverable for compliance-conscious and suitability-focused audiences.

"Done" looks like: *scenario results are directionally sensible (growth-tilted equity-heavy portfolio shows materially worse 2008 result than a conservative one) and roughly reconcile to known historical asset-class behavior.*

Phase 13 — Fama-French Factor Exposure

- Source factor returns from the Ken French Data Library (market, SMB, HML, optionally momentum/quality).
- Run multiple linear regression of excess returns against factor returns (statsmodels) over a chosen lookback window.
- Report R^2 alongside coefficients — a low R^2 means the model isn't explaining much, and hiding that undermines credibility with sophisticated advisers.

"Done" looks like: *regression runs cleanly, produces sensible factor loadings for a known portfolio composition, and reports R^2 alongside coefficients.*

Phase 14 — Monte Carlo Simulation

Deliberately last — depends on the correlation matrix and is the most assumption-heavy module in the build.

- Treat probability-cone-of-outcomes and retirement-readiness as two sub-modules sharing one simulation engine, not one monolithic feature — very different horizon and assumption sensitivity.
- Resolve the source of expected-return/volatility assumptions per asset class (historical average, in-house capital market assumptions, or a licensed third-party CMA set) as a business decision before writing the simulation loop.
- Use Cholesky decomposition of the Phase 10 correlation matrix for correlated random draws — treating asset classes as independent misrepresents diversification and drawdown risk.
- Retirement readiness needs its own cash-flow modeling layer (contributions, withdrawals, timing) on top of the pure value simulation.

- 1,000–10,000 paths is standard and computationally trivial — the harder design question is how you summarize output (percentile bands, probability of success), not the path count.

"Done" looks like: simulated median outcome roughly matches a deterministic compound-growth check using the same assumed mean return, and percentile bands widen sensibly with horizon length.

Phase 15 — Audit Trail Integration Pass

Not a new analytics module — a checkpoint. Walk every module from Phase 1 onward and verify each writes a CalculationRun record: input hash, params, timestamp, output reference, code version. Cheaper to close gaps now than after Pillar 3 is built on top of an audit log with silent holes in it.

6. How This Sets Up Pillars 2 & 3 Without a Rebuild

- **Pillar 2 (suitability & drift):** consumes InvestmentPolicy against the same daily_positions + SecurityMaster taxonomy already used in Phase 7's construction analysis. Drift monitoring is current allocation vs. target vs. tolerance band — no new position/security data model required.
- **Pillar 3 (compliance audit trail):** consumes the CalculationRun log accumulating since Phase 1. The retention/switching-cost moat depends on this history existing from the start — it cannot be credibly backfilled after the fact.